

Security Weakness Report

MAZE Temporal Adventure

Scope: White Box Testing

Dynamic Application Security Testing (DAST) and Static Application Security Testing (SAST)

Date: 03 August 2026

This report lists security weaknesses present in the MAZE Temporal Adventure Game.

Summary

#	Finding	Severity
1	Passwords stored and compared in plaintext	Critical
2	Flat file credential store vulnerable to newline/field injection leading to account takeover	Critical
3	Credentials file (users.txt) contains real looking data	Critical
4	No input sanitization on username or password	High
5	No password strength policy	High
6	No brute-force / rate-limiting protection on login	High
7	Auth bypass	Medium

Security Findings

Cryptographic Failures

Location: UserData.java — saveUser() and checkLogin()

Details: Registration writes the password the user typed straight to disk with no hashing, salting or encryption of any kind. Login reads the same plaintext file and does a direct string comparison.

Evidence (UserData.java)

```
public static void saveUser(String username, String password) {
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(FILE_NAME, true))) {
        bw.write(username + "," + password);
        bw.newLine();
    } catch (IOException e) { e.printStackTrace(); }
}

public static boolean checkLogin(String username, String password) {
    ...
    if (data[0].trim().equals(username) && data[1].trim().equals(password)) {
        return true;
    }
}
```

Impact: Anyone who can read this file, immediately has every user's real password, not just a hash. If any of these people reuse that password elsewhere (email, other sites), those accounts are exposed too.

Injection

Location: UserData.java — saveUser() (no input sanitization)

Details: Registration writes `username + "," + password` followed by a line separator, with zero validation of either field. Because the file format relies on commas to split fields and newlines to split records, a username or password containing those characters is written verbatim and reinterpreted by `checkLogin()` as one or more *additional* records the next time the file is read.

This was verified directly against the project's own UserData class (not a hypothetical) with the small standalone test below.

Evidence:

```
Files.write(Paths.get("users.txt"), "victim,originalSecretPass\n".getBytes());
String maliciousUsername = "attacker\nvictim"; String
maliciousPassword = "hijackedPassword";
UserData.saveUser(maliciousUsername, maliciousPassword);
```

Impact:

The attacker's injected line is parsed by `checkLogin()` as a second, independent "victim,hijackedPassword" record. Logging in as *victim* with the attacker's chosen password now succeeds, leading to a full account-takeover primitive, without ever knowing the victim's real password. The same primitive can be used to pre-plant a lookalike account before a real user has registered, or simply to corrupt the credential store with malformed rows.

Authentication Failures:

1. Improper Input Validation

Location: LoginForm.java, RegisterForm.java, UserData.java

Details: Both forms only check that the fields are non-empty after trimming. There is no length cap, no character allow-list, and no filtering of delimiter / control characters before the values are persisted.

```
if (user.isEmpty() || pass.isEmpty()) {  
    JOptionPane.showMessageDialog(this, "Please enter username and password.");  
    return;  
}
```

Impact: It generally means the credential store's integrity depends entirely on users typing "reasonable" values.

2. Weak Password Requirements

Location: RegisterForm.java

Details: Registration accepts any non-empty password, however short or simple. The shipped users.txt is real evidence of the result: every existing account uses a 3-digit numeric password.

Evidence:

```
abdullah,110  
Saad,123  
saifullah,089  
ahmad,112
```

Impact: 3-digit numeric passwords have only 1,000 possible values means every one of these accounts can be brute force.

3. No Rate-Limiting Protection

Location: LoginForm.java, UserData.checkLogin()

Details: There is no cap on failed login attempts, no delay / backoff and no lockout. checkLogin() can be invoked as fast as the UI can trigger it.

Impact: An attacker with local access to the running app can trivially guess or offline brute-force any account.

Mishandling of Exceptional Conditions

Location: users.txt

Details: The project folder as provided has no. gitignore file at all. The users.txt file sits directly in the project root next to src/ and bin/ and already contains what look like real names and passwords.

Impact: A publicly exposed credentials file leading to full account compromise (verbose errors).

Broken Access Control

Location: Dashboard.java (Level1.java ... Level6.java)

Details: The screen that lists all 6 levels and is to be reachable after a successful login, ships its own standalone entry point that takes a username string and performs no credential check whatsoever:

```
public static void main (String [] args) {  
    SwingUtilities.invokeLater(() -> new Dashboard("Abdullah"). set Visible(true));  
}
```

The Dashboard (String username) constructor itself never calls UserData.checkLogin(), it trusts whatever username string it is handed.

Impact: The login screen is a UI convention not an enforced boundary, anyone who can run a class from this codebase (e.g. java MainGame.Dashboard instead of java App) reaches the full game, leaderboard and dashboard with zero credentials, Realistic severity is tempered here.